

Why did Amazon charged my billing ours for OpenSearch, if my collection is serverless?

Pensou por 15s >

“Serverless” no OpenSearch **não significa “sem cobrança enquanto parado”** — significa **sem você provisionar/gerenciar instâncias**, mas o serviço ainda mantém **capacidade mínima** pronta para atender (e cobra por isso).

Pelo seu print, o que foi cobrado foi **OpenSearch Serverless OCU-hours**:

- **Indexing OCU:** 1.117,5 OCU-hours × **US\$ 0,24** = **US\$ 268,20**
- **Search OCU:** 1.117,5 OCU-hours × **US\$ 0,24** = **US\$ 268,20**
- **Total:** **US\$ 536,40**

Isso bate exatamente com o relatório. Ou seja: sua collection serverless ficou “ativa” e consumindo a **capacidade baseline** (mesmo com tráfego baixo/zero), e a AWS cobra **por hora** essa capacidade separada em **Indexing** e **Search**.

Por que existe custo mesmo sem uso?

No OpenSearch Serverless, a capacidade é medida em **OCUs (OpenSearch Compute Units)**. O serviço mantém OCUs disponíveis para:

- aceitar escrita/indexação quando chegar,
- responder buscas com baixa latência,
- manter o backend operante (encryption, endpoints, coordenação, etc.).

Em muitas configurações, existe um **mínimo efetivo** de OCUs para Search e para Indexing, então “idle” ainda custa.

Por que aparece “OpenSearch Service” no Billing?

A fatura agrupa Serverless e Domains tradicionais sob o guarda-chuva **Amazon OpenSearch Service**. Dentro dele é que aparecem os “usage types” como **SearchOCU** e **IndexingOCU** (que são típicos de Serverless).

Como parar de ser cobrado

1. Delete a collection (se não precisa mais).

Enquanto a collection existir e estiver Active, a cobrança tende a continuar.

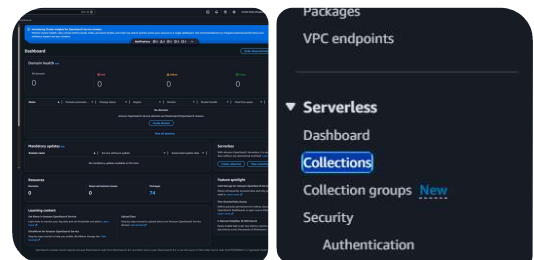
2. Confirme no **Cost Explorer**:

- Service: *Amazon OpenSearch Service*
- Agrupar por **Usage type**
- Você deve ver **SearchOCU-Hours** e **IndexingOCU-Hours**

3. Se você precisa de algo que fique muitas horas “parado”, avalie alternativas:

- um **domain provisionado pequeno** (às vezes sai mais barato para idle contínuo),
- ou até usar outro storage/engine e só subir OpenSearch quando necessário (dependendo do seu caso).

Se você me disser **há quantos dias** essa collection ficou ativa e se tinha ingestão/busca de fato, eu te ajudo a estimar o custo mensal esperado e o jeito mais barato de manter seu RAG rodando.



E se eu mudar de serverless para outra opção, como um cluster? Existe alguma opção que só gaste por tempo processado para RAG hybrid search na Amazon?

Pensou por 23s >

Sim — dá pra sair do **OpenSearch Serverless** e ir pra um **cluster (domain “managed”)**, mas isso **não vai virar “pay-per-request”**. Vai só trocar o *tipo de “always-on”*:

1) Trocar Serverless → Cluster (Managed Domain): o que muda no custo?

Managed Domain (cluster) cobra principalmente por:

- Instance-hours (as instâncias ligadas 24/7)
- EBS storage
- (transferências, snapshots, etc.) Amazon Web Se... +1

Ou seja: você volta a ter “máquinas” rodando o tempo todo. Em workloads pequenos e constantes, pode até sair **mais previsível** (e às vezes mais barato que Serverless), mas **não** é “**pago só quando consulta**”.

2) Existe algo na AWS que seja “só por tempo processado” pra RAG Hybrid Search?

Para **busca híbrida (vetor + keyword/BM25)**, a opção que mais se aproxima do “pago por uso” na AWS hoje é:

Aurora PostgreSQL + pgvector (de preferência Aurora Serverless)

- Você paga por capacidade medida em ACUs, com granularidade por segundo, e ele pode **escalar pra baixo** em períodos de baixa atividade (dependendo do modo/configuração). Amazon Web Se... +1
- E o **Bedrock Knowledge Bases** já suporta **hybrid search** com **Aurora PostgreSQL** (além de OpenSearch). Amazon Web Se...

Mas atenção: mesmo em Aurora Serverless, nem sempre é literalmente “zero custo em idle” em todo cenário; o que você ganha é **escala bem mais fina** e custo bem mais alinhado ao uso do que um cluster OpenSearch sempre ligado.

3) Qual escolha faz sentido pro seu caso (RAG híbrido) quando você quer reduzir idle cost?

Se você quer o *mais “pay-as-you-go” possível*:

Aurora Serverless + pgvector + hybrid search (via Bedrock KB ou app direto).

Amazon Web Se... +2

Se você quer *relevância/recursos de busca e ranking mais “search engine”*:

OpenSearch (managed domain) (mais controle e previsível), mas paga 24/7 por instância.

Amazon Web Se...

Se você quer *simplicidade operacional* (sem mexer com DB) e aceita pagar **baseline**:

OpenSearch Serverless, mas com esse “pisso” de OCUs (que foi exatamente o que te pegou). (A cobrança em OCU-hours é o modelo do Serverless.) Amazon Web Se... +1

Caminho prático pra você agora (pra não repetir a fatura de US\$ 500+)

1. Delete a collection Serverless que não esteja sendo usada (pra cortar o baseline).
2. Para o próximo MVP de RAG híbrido com custo baixo em idle: **Aurora PostgreSQL (Serverless) + pgvector**. Amazon Web Se... +2

Se você me disser:

- volume de documentos (GB) e tamanho médio,
 - QPS (consultas por segundo) esperado,
 - e se precisa de filtros/aggregations pesadas,
- eu te digo qual dos 3 (Aurora+pgvector vs OpenSearch managed vs continuar no serverless com limites) tende a ficar **mais barato** e **menos dor de cabeça** pro seu padrão de uso.

        Fontes

No caso, qual é o mais barato de todas essas alternativas?

Pensou por 14s >

Depende **muito** do seu padrão de uso (quantas horas “parado” vs picos), mas dá pra responder de forma prática:

O mais barato, no geral

1) Se o seu RAG fica a maior parte do tempo “idle” (poucas consultas por dia / MVP / dev):

 **Aurora Serverless v2 (PostgreSQL) + pgvector** tende a ser o mais barato, porque cobra por **ACU por segundo** e pode **escalar pra baixo** em baixa demanda. Amazon Web S...

E o Bedrock Knowledge Bases já suporta **hybrid search** com **Aurora PostgreSQL**.

Amazon Web Se... +1

Em termos de “só pagar quando usa”, é o que mais chega perto entre as opções AWS para hybrid search.

2) Se o seu RAG tem uso constante (muitas consultas/ingestão durante o dia todo):

✅ **OpenSearch Managed Domain (cluster)** costuma ser o mais barato/eficiente por dólar quando você consegue **right-size** e o uso é previsível — mas é **always-on** (cobra por hora de instância + storage). Amazon Web Se... +1

3) OpenSearch Serverless (o seu caso)

❌ Geralmente é o **mais caro** quando a collection fica “ligada” sem tráfego, porque há cobrança contínua por **OCU-hours** (Search e Indexing). Coralogix +1

Seu print (US\$ 0,24/OCU-h em Search + US\$ 0,24/OCU-h em Indexing) é exatamente esse modelo. Coralogix

Regra de bolso (bem direta)

- Pouco uso / muitas horas parado → **Aurora Serverless v2 + pgvector** (mais barato na maioria dos MVPs) Amazon Web Se...
- Uso constante → **OpenSearch Managed Domain** (mais barato que serverless em steady-state, na prática) Amazon Web Se... +1
- **Serverless OpenSearch** → só faz sentido quando você quer operação simples + escala elástica, e aceita pagar a capacidade de Search/Indexing em OCU-hour.

Coralogix +1

Se você me disser (mesmo por alto) **consultas/dia**, **tamanho do corpus** e se tem **ingestão contínua** ou só batch, eu cravo qual fica mais barato no seu caso e passo um desenho de arquitetura enxuto.

     ...  Fontes

Nosso caso é bem baixo, algumas dezenas de consultas em toda a semana, só quando ligamos o projeto para testar.

Nesse perfil (dezenas de consultas **na semana**, quase sempre **idle**), a opção **mais barata na AWS** quase sempre é:

✓ Aurora PostgreSQL + pgvector (preferencialmente Aurora Serverless v2)

Porque você foge do “piso” de custo de **OpenSearch Serverless** (OCU sempre rodando) e também do “sempre ligado” de **cluster OpenSearch** (instance-hours 24/7). Aurora Serverless v2 cobra por capacidade em **ACUs** com granularidade fina, então tende a ficar bem mais alinhado ao seu uso esporádico.

E como você quer **hybrid search**, dá pra fazer de dois jeitos:

- **Bedrock Knowledge Bases** com hybrid search usando **Aurora PostgreSQL** (se você quiser conveniência).
- Ou **na mão** no app: BM25-like via `tsvector / pg_trgm` + vetor via `pgvector` (mais controle, menos “mágica”).

O que eu NÃO usaria no seu cenário

✗ OpenSearch Serverless (o que te cobrou US\$ 500+)

Serverless no OpenSearch tem custo contínuo em **SearchOCU** + **IndexingOCU**. Para ambiente que fica parado, costuma ser a pior combinação custo/uso.

✗ OpenSearch cluster (Managed Domain)

Mesmo o menor cluster “baratinho” vira custo fixo 24/7 (instância + EBS), e você vai pagar por semana inteira pra fazer poucas consultas.

Recomendações práticas pra você gastar quase nada em dev/test

1. **Destruir recursos quando não estiver usando**
 - Se insistir em OpenSearch (qualquer modo), delete ao final do teste.
2. **Se você precisa manter sempre pronto, mas com uso baixo**
 - Aurora Serverless v2 + `pgvector`.
3. **Se o dataset for pequeno (ex.: poucos milhares de chunks) e é só teste local**
 - Rodar FAISS/SQLite localmente (custo AWS ~ zero). Se quiser “100% AWS”, dá pra por isso em uma VM pequena e ligar só quando testar (mas aí vira outra história).

Se você me disser:

- tamanho do corpus (quantos PDFs / chunks / embeddings),
 - modelo de embedding (dimensões),
 - e se você precisa de filtros por metadados (tipo `tenant_id`, datas, etc.),
- eu te passo uma arquitetura mínima e um “checklist” de custo (o que cria e o que NÃO criar) pra não tomar outra pancada no billing.



E porque preferencialmente o Aurora Serverless v2?

Porque, no seu cenário (quase sempre **idle**), o Aurora Serverless v2 costuma ser o que **melhor “encaixa” custo ↔ uso** entre as opções AWS que você está considerando.

1) Escala muito mais “fina” e contínua

O Serverless v2 ajusta capacidade em **incrementos pequenos (ACUs)** e de forma bem mais contínua do que ter um cluster “sempre ligado”. Isso evita você pagar uma instância inteira 24/7 só pra fazer alguns testes na semana.

2) Menos penalidade de “ficar parado”

OpenSearch Serverless te cobrou porque há **capacidade mínima** de Search/Indexing (OCUs) rodando enquanto a collection existe/está ativa. Já no Aurora Serverless v2 a cobrança acompanha melhor a capacidade realmente necessária (e você consegue manter bem baixo).

3) Continua sendo Postgres “de verdade” (bom pra dev e produção)

Você ganha:

- SQL, transações, migrações, tooling comum
- filtros/joins por metadados de RAG (muito útil)
- pgvector para embeddings

E ainda pode plugar no **Bedrock Knowledge Bases com hybrid search em Aurora PostgreSQL** se quiser conveniência.

4) v2 evita alguns “sustos” do Serverless antigo

Aurora Serverless v1 (o antigo) tinha um modelo de liga/desliga mais “grosso” e pode introduzir mais latência em retomadas. O v2 foi desenhado para ser mais suave/contínuo, o que costuma ser melhor para workloads intermitentes de API (como RAG).

Resumo prático pro seu caso:

Se você vai rodar *poucas dezenas de queries por semana*, Aurora Serverless v2 + pgvector tende a ser o “modo econômico” que ainda te dá **hybrid search** e um backend bem padrão.

Se você quiser, me diga a região (ex.: us-west-2), e uma estimativa de **quantos chunks** você pretende armazenar (1k? 100k? 1M?). Aí eu te digo se vale mais Aurora Serverless v2, RDS Postgres "normal" pequeno, ou até algo local/temporário pra dev.



kubernetes-sagemaker-jo... Arquivo	sagemaker-roles.md Arquivo
kubernetes-sagemaker-o... Arquivo	connect-sagemaker.md Arquivo
	automating-sagemaker-w... Arquivo

A fonte será a pasta sagemaker docs:

Documentos: 336 arquivos (todos .md)

Tamanho total: ~1.42 MB (1,491,966 bytes)

Estimativa de chunks

Com um tamanho típico de chunk de ~1000 caracteres (~250-300 tokens), os 336 documentos (~1.49 MB de texto) devem gerar algo em torno de 1k a 2k chunks

Região: us-east-1

Estimativa de chunks: ~1k a 2k chunks

Estou mandando os 5 maiores arquivos dessa pasta

Pensou por 21s >

Com esse volume (1k–2k chunks) e uso (dezenas de consultas/semana só em teste), o mais barato é **não** manter nenhum "motor de busca" gerenciado ligado 24/7.

Opção mais barata na AWS (quase "só paga quando usar")

✓ Lambda (ou ECS Fargate on-demand) + índice local (FAISS + BM25) + arquivos no S3

Ideia: você pré-processa 1 vez (chunks + embeddings + um índice BM25 simples) e salva no S3.

Quando você quer testar, uma **Lambda**:

1. carrega os artefatos do S3 (ou reaproveita cache em /tmp),
2. roda **hybrid** (BM25 + cosine),
3. retorna top-K pro seu RAG.

Por que fica tão barato?

- Sem cluster/OCU/instância ligada.
- Você paga **por invocação e tempo de execução** (e o S3 é centavos pra 1–2 MB + embeddings).
- 1–2k vetores cabem fácil em memória.

Trade-offs: cold start e um pouco mais de trabalho pra empacotar o índice; mas pra “ambiente de teste” é perfeito.

Se você quer o *mínimo custo possível*, essa é a resposta.

Segunda mais barata (se você quer algo “sempre disponível”, mas econômico)

✓ Aurora PostgreSQL (de preferência Serverless v2) + pgvector

É a alternativa “gerenciada” que costuma ficar mais barata **quando o uso é baixo**, porque você evita:

- o **piso de custo** do OpenSearch Serverless (OCU),
- e o **always-on** de um cluster OpenSearch.

Trade-off: ainda existe custo base do banco + storage, mas geralmente bem menor e mais proporcional que OpenSearch para o seu perfil.

As que eu evitaria no seu perfil

✗ OpenSearch Serverless

Foi o que te mordeu: cobra **SearchOCU + IndexingOCU por hora** mesmo com quase nada acontecendo.

❌ OpenSearch “cluster” (Managed Domain)

É literalmente **instância ligada 24/7** (mais EBS). Pra testar poucas vezes por semana, quase sempre é dinheiro queimando.

Recomendação direta pro seu caso

- Se vocês topam **um pouco mais de engenharia: Lambda + FAISS/BM25 + S3** (mais barato disparado).
- Se vocês querem **menos custom e ainda barato: Aurora Serverless v2 + pgvector**.

Se você me disser em que linguagem vai rodar o backend (Python/.NET) e qual embedding (dimensão), eu te passo um esqueleto bem direto do pipeline “build index → subir no S3 → Lambda query hybrid”.



Estamos usando python com aioboto3 e fastapi, Amazon Titan Embed Text v2 (amazon.titan-embed-text-v2:0) via Bedrock, com 1024 dimensões.

Pensou por 6s

Perfeito — com **Python + FastAPI, aioboto3 e Titan Embed Text v2 (1024 dims)**, você está no cenário ideal pra uma arquitetura **quase “pague só quando rodar”**.

O caminho mais barato (e simples o bastante) pro seu volume

✅ **“Índice offline no S3” + “consulta on-demand” (Lambda ou Fargate) + FastAPI**

Como você tem só **1k–2k chunks**, dá pra manter tudo super leve:

Tamanho real dos vetores (pra você ter noção)

- 2.000 vetores × 1024 dims × 4 bytes (float32) ≈ **8 MB**
- Se salvar float16: ≈ **4 MB**

Isso é minúsculo. Dá pra carregar em memória sem drama.

Implementação recomendada (hybrid de verdade, baratinho)

A) Index-time (rodar 1x quando atualizar docs)

1. **Chunking** dos `.md` (ex.: ~800–1200 chars, overlap 100–200)
2. **Embeddings** via Bedrock Titan v2 (1024 dims)
3. Criar artefatos:
 - `vectors.npy` (matriz Nx1024)
 - `meta.jsonl` (`doc_id`, `path`, `chunk_text`, `offsets`, etc.)
 - **BM25** (recomendo `rank_bm25`) ou **SQLite FTS5** (já vem no SQLite e fica bem

≡ ChatGPT 5.2 Thinking ▾



B) Query-time (rodar só quando vocês testam)

1. Embedding do query (Titan v2)
2. Score lexical (BM25/FTS)
3. Score vetorial (cosine / dot product)
4. Combina (ex.: $\text{score} = 0.6 * \text{vec} + 0.4 * \text{lexical}$) + rerank opcional
5. Retorna top-K chunks pro seu prompt

Onde rodar pra pagar o mínimo possível

Opção 1 (mais barata): AWS Lambda

- Ideal se suas queries são poucas e “liga só pra testar”.
- Você paga por invocação/tempo.
- **Atenção prática:** se você for carregar ~10–30 MB do S3 por request, pode ficar lento.

A solução:

- cache em `/tmp` + reuse do container quente
- ou “warm-up” manual quando for testar

Opção 2 (muito boa e ainda barata): ECS Fargate “on-demand”

- Você **sobe um task** quando quer testar e derruba depois.
- Custa mais que Lambda por hora, mas:
 - cold start menor
 - runtime mais previsível
 - você pode manter sua FastAPI como está, sem adaptar para Lambda
- Para “ligar só quando testar”, costuma ficar bem econômico.

Entre "Lambda vs Fargate": se você quer zero fricção com FastAPI, **Fargate on-demand** costuma ser o sweet spot.

Por que eu NÃO usaria OpenSearch no seu caso (mesmo managed)

Com 1k–2k chunks, OpenSearch é "canhão pra matar mosquito".

- **Serverless**: tem piso de OCU e você já viu o estrago.
- **Managed domain**: paga 24/7 por instância + EBS.

Você consegue hybrid search melhor custo/benefício com **BM25/FTS + vetores em memória**.

Stack concreta (bem pé-no-chão) pro seu projeto

- **S3**: guarda docs e artefatos do índice
 - **FastAPI**:
 - endpoint `/search` (hybrid)
 - endpoint `/reindex` (opcional, protegido)
 - **Bedrock Titan v2**: embeddings do query (e do index, quando atualizar)
 - **BM25**: `rank_bm25` ou **SQLite FTS5** (eu gosto muito do FTS5 porque evita "índice na RAM" e é rápido/estável)
 - **Vetores**: `numpy` + cosine/dot (FAISS é opcional nesse tamanho)
-

Uma recomendação extra que economiza MUITO

Não re-embedde tudo sempre.

- Use hash do chunk (`sha1(texto)`) e só gera embedding para chunks novos/alterados.
 - Com docs mudando pouco, o custo Bedrock cai pra quase nada.
-

Se você quiser, eu já te mando um esqueleto de pastas + código (indexer e query hybrid) em Python, no estilo:

- `indexer/build_index.py` (lê `.md`, chunk, chama Bedrock, gera artifacts e sobe S3)
- `api/app.py` (FastAPI, baixa artifacts do S3 no startup/cache, endpoint `/search`)

Só me diga: vocês vão rodar isso **local**, **EC2**, **Lambda**, ou **Fargate**? (eu consigo adaptar o mesmo core pros 3).

